Trabalho Prático 2
DevOps

Vinicius Trindade Dias Abel
Matrícula: 2020007112

Universidade Federal de Minas Gerais (UFMG)
Belo Horizonte – MG – Brasil

viniciustda@ufmg.br

1. Introdução

O objetivo deste trabalho foi projetar, implementar e automatizar o deploy de uma aplicação de recomendação de playlists. A arquitetura foi baseada em microserviços, separando a API REST (criada em Flask) do processo de treino do modelo de Machine Learning (usando FP-Growth).

O foco principal foi a automação do ciclo de vida da aplicação (MLOps), que foi alcançada usando **Docker** para containerização, **Kubernetes (K8s)** para orquestração, e **ArgoCD** para a entrega e integração contínua (CI/CD) baseada num repositório **Git**. O resultado é um sistema capaz de ser atualizado (seja o seu código, configuração ou modelo de ML) automaticamente, sem intervenção manual e sem downtime.

2. Discussão do Pipeline de CI/CD e Casos de Teste

O pipeline de CI/CD foi construído com o ArgoCD monitorando um repositório Git. Qualquer alteração (via git push) nos arquivos de configuração *.yaml* inicia automaticamente um processo de sincronização, que atualiza os recursos no cluster Kubernetes.

Para validar este pipeline, foram realizados três testes de CI/CD.

2.1 Teste 1: Atualização da Configuração (Réplicas)

- **Ação:** O arquivo *deployment.yaml* foi alterado no Git, mudando o número de réplicas de replicas: 1 para replicas: 2.
- **Resultado Esperado:** Era esperado que o ArgoCD detectasse a mudança de configuração e ordenasse o Kubernetes criar um segundo pod da API, sem interromper o serviço existente.
- **Resultado Real:** O ArgoCD sincronizou a mudança em segundos. Na imagem 1 abaixo é possível obaservar dois pods da API no estado Running, provando que a atualização automática funcionou.

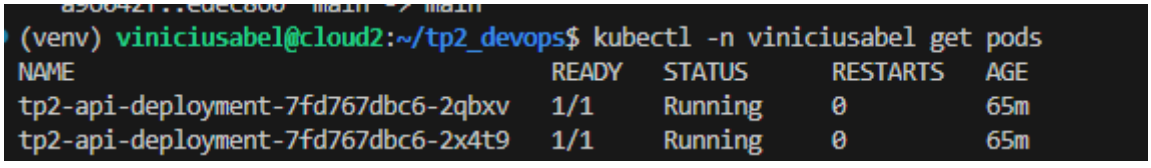


Imagem 1: Atualização do número de réplicas

2.2 Teste 2: Atualização do Código (Nova Versão da Imagem)

- **Ação:** O código da API (*app.py*) foi alterado (mudando da versão 2.0 para 2.1). Uma nova imagem Docker (viniciustda/tp2-api:2.1) foi construída e enviada para o DockerHub. O arquivo *deployment.yaml* no Git foi atualizado para usar esta nova *tag* de imagem.
- **Resultado Esperado:** Esperava-se que o Kubernetes realizasse uma atualização contínua, substituindo os pods antigos (v2.0) pelos novos (v2.1) um de cada vez, garantindo zero downtime.
- **Resultado Real:** A atualização foi um sucesso. O loop de curl (cliente de monitorização) mostrou as respostas JSON mudando de “version":"2.0" para "version":"2.1" sem falha de conexão, como mostra a imagem 2.

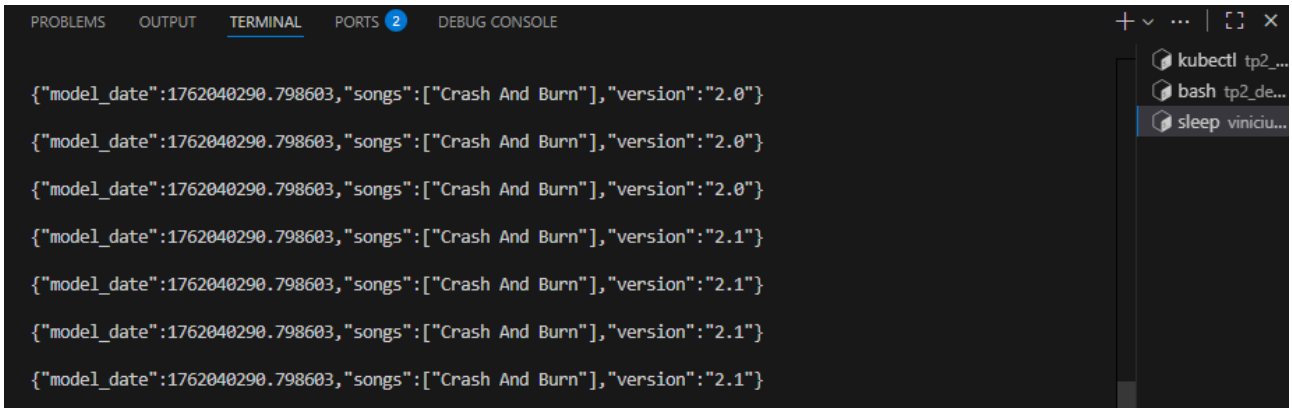


Imagem 2: Atualização de versão

2.3 Teste 3: Atualização do Dataset (Novo Modelo de ML)

- **Ação:** O arquivo *job.yaml* no Git foi alterado para usar o dataset *2023_spotify_ds2.csv* (em vez do *2023_spotify_ds1.csv*). O nome do Job também foi alterado (para tp2-ml-job-ds2) para forçar a sua recriação.
- **Resultado Esperado:** Era esperado que o ArgoCD corresse o novo Job para gerar um novo arquivo pickle. A API, por sua vez, deveria detectar automaticamente o novo modelo (sem reiniciar) e começar a usá-lo.
- **Resultado Real:** O ArgoCD correu o novo Job (conforme mostra a imagem 3 abaixo). Durante este tempo, a API continuou usando o modelo antigo. Após a conclusão do Job, o loop de curl (imagem 4) mostrou o model_date na resposta JSON sendo atualizado automaticamente para um timestamp mais recente. Mostrando que o recarregamento dinâmico do modelo pela API funcionou como esperado.

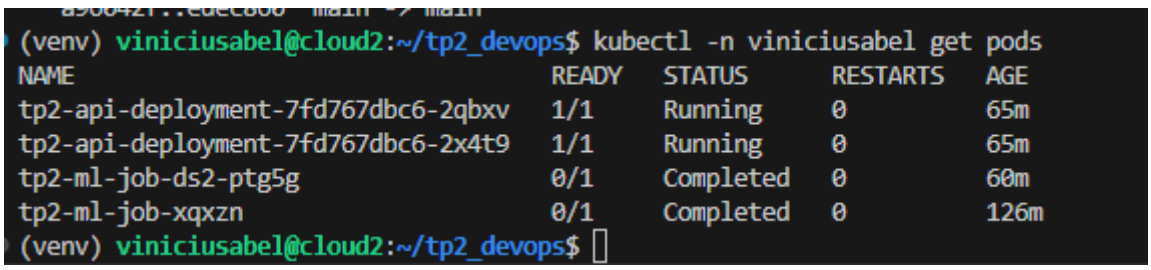


Imagem 3: Atualização do Job

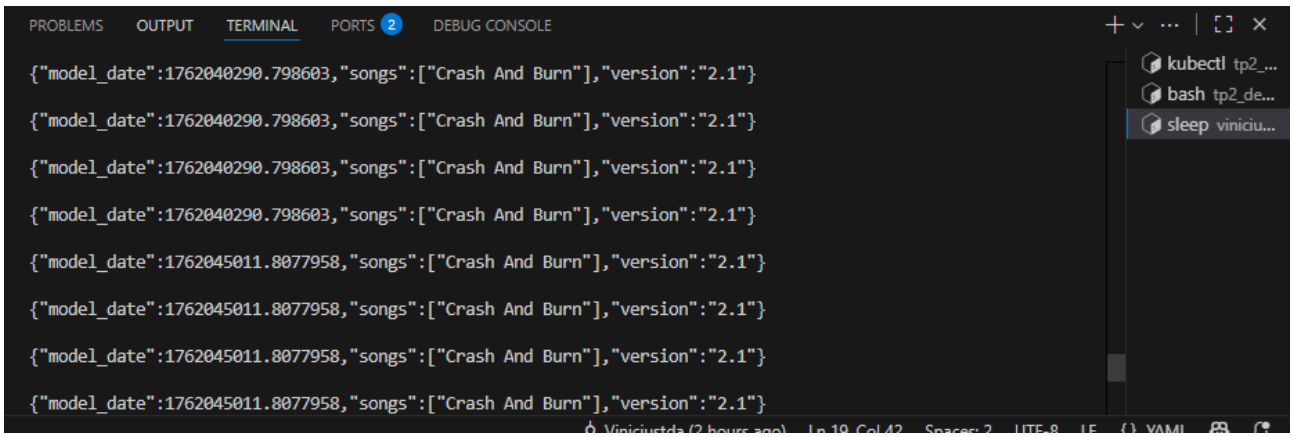


Imagem 4: Atualização do modelo

3. Tempo de Deployment e Análise de Downtime

Com o loop de curl contínuo, foi possível observar que a aplicação não ficou inativa em momento algum em todos os três cenários de teste.

- **Ao atualizar as réplicas (Teste 1):** A atualização (scale-up) foi quase instantânea. O segundo pod ficou em Running rapidamente após o ArgoCD detectar o git push, sem afetar o pod existente.
- **Ao atualizar o código (Teste 2):** A estratégia de rolling update do Kubernetes fez com que não houvesse tempo inativo. Foi observado que o K8s só terminava um pod antigo (v2.0) depois de o pod novo (v2.1) estar em Running e responder pedidos, garantindo uma transição suave.
- **Ao atualizar o dataset (Teste 3):** Não houve downtime de serviço. A API continuou usando o modelo antigo (ds1) enquanto o novo Job (ds2) corria. A atualização do modelo para o utilizador final foi instantânea no momento do primeiro pedido curl após a conclusão do Job.

4. Detecção de Mudanças no Modelo (API Back-end)

A detecção de mudanças no modelo é uma funcionalidade central da API (*app.py*) e é importante para o Teste 3. A API não precisa de ser reiniciada para carregar um novo modelo.

A lógica de detecção funciona da seguinte forma:

1. A aplicação Flask mantém duas variáveis globais: app.rules (o modelo carregado) e app.model_timestamp (a data de modificação desse modelo).
2. Na inicialização, a função load_model() é chamada para carregar o que estiver no disco (mesmo que seja nada).
3. A cada pedido POST recebido na rota /api/recommend, a função primeiro verifica a data de modificação atual do arquivo no volume (current_timestamp = os.path.getmtime(MODEL_PATH)).
4. É feita uma comparação: se o timestamp do arquivo no disco for mais recente que o timestamp do modelo em memória (current_timestamp > app.model_timestamp), a função load_model() é chamada novamente.
5. Esta função recarrega o novo arquivo pickle para a variável app.rules e atualiza o app.model_timestamp.
6. A resposta é gerada usando as regras atualizadas.

5. Obtenção de Novos Datasets (Container ML)

O container de ML (tp2-ml) foi desenhado para ser agnóstico aos dados e não contém nenhum dataset na sua imagem Docker, seguindo as instruções.

A obtenção do dataset é gerida pelo Kubernetes através de dois mecanismos definidos no job.yaml:

1. **Montagem de Volume (hostPath):** O arquivo *job.yaml* instrui o Kubernetes a montar o diretório hostPath da VM (o /home/datasets/spotify, que contém todos os arquivos .csv) dentro do container, no caminho virtual /data/.
2. **Variável de Ambiente:** O mesmo *job.yaml* (que está no Git e é monitorizado pelo ArgoCD) define uma variável de ambiente chamada DATASET_PATH (ex: /data/2023_spotify_ds1.csv). O script *generate_rules.py* lê esta variável de ambiente (os.environ.get('DATASET_PATH')) para saber qual arquivo exato deve abrir. Para atualizar o dataset (como no Teste 3), apenas foi necessário alterar esta variável no *job.yaml* e fazer o git push, desencadeando todo o pipeline de retreino.

6. Conclusão

Este trabalho demonstrou com sucesso a criação de um pipeline MLOps completo e funcional. Ao usar o Git, o ArgoCD automatizou todo o ciclo de vida da aplicação, desde a implementação da API até ao retreino do modelo de ML, sem intervenção manual. Os testes provaram que a arquitetura é resiliente, capaz de ser atualizada (em configuração, código ou dados) com zero downtime, validando os princípios fundamentais do DevOps e MLOps.